Object Oriented Programming

LAB-01 Solutions

Task 01

```
#include <iostream>
using namespace std;

// returns a new array of double size
// first half = original elements, second half = 2 x original elements
int* resizeArray(int inArray[], const int size)
{
    int* newArray = new int[size * 2];

    for (int i = 0; i < size; i++)
    {
        newArray[i] = inArray[i];
        newArray[i + size] = inArray[i] * 2;
    }

    return newArray;
}

int main()
{
    int size;
    cout << "Enter size of the array: ";
    cin >> size;

    int* arr = new int[size];

    cout << "Enter elements of array: ";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    int* result = resizeArray(arr, size);

    cout << "Resized array: ";
    for (int i = 0; i < size * 2; i++)
    {
        cout << result[i];
        if (i != size * 2 - 1)
            cout << ", ";
    }
    cout << endl;

    // free memory
    delete[] arr;
    delete[] result;

    return 0;
}
```

Task 02

```
#include <iostream>
using namespace std;

// checks if all elements outside the main diagonal are zero
bool isDiagonal(int** matrix, int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i != j && matrix[i][j] != 0)
                return false;
        }
    }
    return true;
}

int main()
{
    int n;
    cout << "Enter size of the square matrix: ";
    cin >> n;

    // create 2D array dynamically
    int** matrix = new int*[n];
    for (int i = 0; i < n; i++)
    {
        matrix[i] = new int[n];
    }

    cout << "Enter the elements of the matrix:" << endl;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << "Element [" << i << "][" << j << "]: ";
            cin >> matrix[i][j];
        }
    }

    cout << "\nMatrix:" << endl;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }

    if (isDiagonal(matrix, n))
        cout << "\nThe matrix is a diagonal matrix." << endl;
    else
        cout << "\nThe matrix is NOT a diagonal matrix." << endl;

    // free memory
    for (int i = 0; i < n; i++)
    {
        delete[] matrix[i];
    }
    delete[] matrix;

    return 0;
}
```

Task 03

```
#include <iostream>
#include <cstring>
using namespace std;

// returns a new string containing the first 'count' characters of target
char* left(const char* target, const int count)
{
    char* result = new char[count + 1];

    for (int i = 0; i < count; i++)
    {
        result[i] = target[i];
    }
    result[count] = '\0';

    return result;
}

// de-allocates the memory pointed by p
void Destructor(char* p)
{
    delete[] p;
}

int main()
{
    char str[30];
    int count;

    cout << "Enter a string: ";
    cin.getline(str, 30);

    cout << "Enter count: ";
    cin >> count;

    if (count < 0 || count > (int)strlen(str))
    {
        cout << "Count is out of range!" << endl;
    }
    else
    {
        char* sub = left(str, count);
        cout << "Left substring: " << sub << endl;
        Destructor(sub);
    }

    return 0;
}
```

Task 04

```
#include <iostream>
using namespace std;

int findMin(int inArray[], const int size)
{
    int min = inArray[0];
    for (int i = 1; i < size; i++)
    {
        if (inArray[i] < min)
            min = inArray[i];
    }
    return min;
}

int findMax(int inArray[], const int size)
{
    int max = inArray[0];
    for (int i = 1; i < size; i++)
    {
        if (inArray[i] > max)
            max = inArray[i];
    }
    return max;
}

int computeAverage(int inArray[], const int size)
{
    int sum = 0;
    for (int i = 0; i < size; i++)
    {
        sum += inArray[i];
    }
    return sum / size;
}

int computeMedian(int inArray[], const int size)
{
    // make a copy so the original array stays unchanged
    int* temp = new int[size];
    for (int i = 0; i < size; i++)
    {
        temp[i] = inArray[i];
    }

    // simple bubble sort (ascending)
    for (int i = 0; i < size - 1; i++)
    {
        for (int j = 0; j < size - i - 1; j++)
        {
            if (temp[j] > temp[j + 1])
            {
                int t = temp[j];
                temp[j] = temp[j + 1];
                temp[j + 1] = t;
            }
        }
    }

    int median;
    if (size % 2 == 1)
        median = temp[size / 2];
    else
        median = (temp[size / 2 - 1] + temp[size / 2]) / 2;

    delete[] temp;
    return median;
}

int main()
{
    int size;
    cout << "Enter the number of students: ";
    cin >> size;

    while (size <= 0)
    {
        cout << "Number of students must be greater than 0. Enter again: ";
        cin >> size;
```

```
    }

    int* marks = new int[size];

    for (int i = 0; i < size; i++)
    {
        cout << "Enter marks of student " << i + 1 << ": ";
        cin >> marks[i];

        // negative marks are not allowed
        while (marks[i] < 0)
        {
            cout << "Marks cannot be negative. Enter again: ";
            cin >> marks[i];
        }
    }

    cout << "\nMaximum marks: " << findMax(marks, size) << endl;
    cout << "Minimum marks: " << findMin(marks, size) << endl;
    cout << "Average marks: " << computeAverage(marks, size) << endl;
    cout << "Median marks : " << computeMedian(marks, size) << endl;

    delete[] marks;
    return 0;
}
```